Actividad 10. Esquema Algorítmico Divide y Vencerás

Objetivo

Resolver problemas que siguen el esquema algorítmico divide y vencerás.

Procedimiento

1. Ver el vídeo o leer la presentación sobre esquemas algorítmicos que están disponibles en Moodle, Tema 4/Sección 4.3 Esquema Algorítmico: Divide y vencerás/ Recursos didácticos.
2. Resolver los ejercicios que se proponen en esta actividad, para reforzar el esquema de vuelta atrás. Para ello haz uso de la clase DivideAndConquerAlgorithm situada en el paquete **es.uvigo.esei.aed2.activity10**, la cual contiene las cabeceras de los métodos.
3. Para probar que la implementación realizada funciona correctamente se proporcionan la clase de prueba DivideAndConquerAlgorithmTestCase.

Evaluación

Estos contenidos serán evaluados mediante una prueba individual el 19 de enero de 2026.

Tiempo estimado

3 horas.

Ejercicios para resolver en clase

1. Un problema que sigue la estrategia **divide y vencerás** es encontrar el *k-ésimo* menor elemento de un array. Obviamente, se puede realizar ordenando el array, pero es de esperar que selección sea un proceso más rápido, al solicitarse menor información. Haciendo un pequeño cambio en el algoritmo *quicksort*, se puede resolver el problema de selección en *tiempo lineal*, en promedio. Llamamos a este algoritmo selección rápida. Los pasos a realizar son los siguientes:

   Siendo "array", la estructura donde se busca el k menor elemento.

   - Si el número de elementos en "array" es 1, entonces presumiblemente *k* también es 1, por lo que se puede devolver el único elemento en "array".
   - En otro caso, elegir un elemento *v* de "array" como pivote:

- Hacer una partición de "array" – {*v*} en "arrayIzq" y "arrayDer", exactamente igual a como se hacía en el *quicksort*.
- Si *k* es menor o igual que el número de elementos en "arrayIzq", entonces el elemento que estamos buscando debe estar en "arrayIzq", por lo que se llama recursivamente a *seleccionRapida*(arrayIzq, *k*). Si *k* es exactamente igual a uno más que el número de elementos de "arrayIzq", entonces el pivote es el *k-ésimo* menor elemento y se puede devolver como respuesta. En otro caso, el *k-ésimo* menor elemento estará en "arrayDer". De nuevo podemos hacer una llamada recursiva y devolver el resultado obtenido.

Utilizando los siguientes métodos privados, implementa el algoritmo de selección rápida.

```
public static int quickSelection (int [] array, int k_minor, int i, int j)
```

// Selecciona un elemento pivote: entre dos elementos diferentes devuelve el elemento mayor.

```
private static int searchPivotPosition
(int [] aux, int inicio, int fin)
{
int primer = aux[inicio];
int k = inicio + 1;

while (k <= fin)
{
   if (aux[k] > primer) {
            return k;
   }
   else if (aux[k] < primer) {
                return inicio;
   }
   else {
        k++;
   }
}
return -1;
}
```

//Divide el array en menores que pivote, y mayores e igual a pivote

// Devuelve la posición del primer elemento de los mayores o iguales a pivote (es decir el número de elementos que hay en la partición de la izquierda).

```
private static int partition (int []
aux, int inicio, int fin, int pivote)
{
  int derecha = inicio;
  int izquierda = fin-1;
  while (derecha <= izquierda)
  {
   while (aux[derecha] < pivote) {
                derecha++;
   }
   while (aux[izquierda] >= pivote) {
                izquierda--;
   }

   if (derecha < izquierda)
      intercambiar (aux, derecha, izquierda);
   }

   return derecha;
  }

private static void exchange (int
[] aux, int i, int j){
    int temp = aux[i];
    aux[i] = aux[j];
    aux[j]=temp;
```

2. El de las Torres de Hanoi es un juego matemático consistente en mover unos discos de una torre a otra. En este juego, se trata de pasar un número de discos (típicamente, con tres existe una dificultad suficiente como para plantearlo como un pasatiempo), de un poste de origen (el primero, más a la izquierda) a un poste de destino (el tercero, a la derecha), utilizando como poste auxiliar el del medio. Sólo se puede mover un disco de cada vez, y nunca poner un disco sobre un segundo que sea de menor diámetro que el primero. Así, al comienzo del juego todos los discos están apilados en el primero (el de la izquierda), cada disco se asienta sobre otro de mayor diámetro, de manera que, tomados desde la base hacia arriba, su tamaño es decreciente. El objetivo, como ya se ha dicho, es mover uno a uno los discos desde el poste A (origen) al poste C (destino) utilizando el poste B como auxiliar, para lo cual se puede emplear una técnica *divide y vencerás*.

3. Dado un **array ordenado ascendentemente** de números enteros X, diseñe un algoritmo que permita determinar si existe un índice *i* tal que X[i] = i. Si existe devuelve el índice, si no existe devuelve -1. Implementa el método **numberExists,** siguiendo el esquema de **divide y vencerás** con una complejidad de O(log n).

Dado el siguiente array, devolvería el índice 2.

-3	0	2	9	10	12	15
0	1	**2**	3	4	5	6

Dado el siguiente array, devolvería el índice 4.

-3	0	1	2	4	12	15
0	1	2	3	**4**	5	6

Dado el siguiente array, devolvería el índice -1.

-3	0	1	2	10	12	15
0	1	2	3	4	5	6

```
public static int numberExists (int[] array, int inicio, int fin)
```

4. Sea "v" un array de enteros positivos que se ajusta al perfil de una curva cóncava, es decir, existe una única posición *K* en el vector tal que:

   a) Los elementos a la izquierda de *K* están ordenados descendentemente.

   b) Los elementos a la derecha de *K* están ordenados ascendentemente.

En el ejemplo, K estaría en la posición 6.

						K			
9	7	5	4	3	2	1	3	4	7

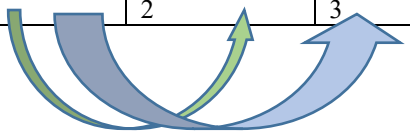
Implementa el método `buscarPosK`, que se indica a continuación, de forma eficiente (complejidad *O(log n)*) para que determine dicha posición *K*. Puedes considerar que el array está correctamente ordenado. En caso de que el array no se ajuste al perfil de curva cóncava devolverá -1 (esto es, si los elementos están todos ordenados de mayor a menor, o si están todos ordenados de menor a mayor). Implementa este algoritmo siguiendo una estrategia de **divide y vencerás**.

```
public static int searchPositionK (int[] v, int start, int end)
```

5. Muchos sitios web intentan comparar las preferencias de un usuario con las de otro, para ello un usuario establece un ranking de n productos, y la aplicación web consulta su base de datos para encontrar usuarios con gustos similares.
   Un algoritmo puede medir la similitud de nuestras preferencias contando el número de inversiones entre dos rankings. Considerando que se valoran *n* productos, el ranking de Ana está ordenado de menor a mayor preferencia (asignando los valores de 1-n en orden) y Juan califica esos mismos productos con un valor entre 1-n sin repetir valores (por lo tanto, si los gustos entre los dos no coinciden, los valores de Juan no estarán ordenados). Se considera que existe una inversión entre los gustos de Ana y Juan, si:
   - Ranking Ana: 1, 2, 3, ..., n
   - Ranking Juan: $a_1, a_2, \ldots, a_n$
   - Si i, j, $a_i, a_j \in \{1\text{-}n\}$, están invertidos si $i<j$ pero $a_i > a_j$
   - Como la condición de que $i<j$ siempre es cierta, es suficiente con comparar si $a_i > a_j$

	ProductoD	ProductoB	ProductoA	ProductoC	ProductoE
Ana	1	2	3	4	5
Juan	1	4	2	3	5

Inversiones: (4-2), (4-3)



Esto es, cuantas menos inversiones existan entre dos rankings, más similares serán las preferencias de los usuarios representados por esos rankings.
Un algoritmo de fuerza bruta compararía las preferencias de los usuarios probando todos los pares posibles de productos $O(n^2)$. Sin embargo, se puede implementar un algoritmo más eficiente, O (n log n), siguiendo el esquema **Divide y vencerás**. El algoritmo sigue los siguientes pasos:

- **División**. Dividir las preferencias de los productos en dos mitades y contar recursivamente el número de inversiones en cada mitad.
- **Combinación**. Asumiendo que cada mitad está ordenada, se contabilizan las inversiones en las que ai y aj están en mitades diferentes. Se mezclan las dos mitades ordenando los elementos. Llamada al método **mergeCount**.
- **Devolver la suma de las 3 cantidades**. (Inversiones de la primera mitad, inversiones de la segunda mitad, inversiones entre los elementos de las dos mitades)

Implementa el método **inversiones**, utiliza el método privado **mergeCount** que realiza la combinación y conteo de inversiones entre elementos de diferentes mitades.
Considerando que el ranking de uno de los usuarios está ordenado de menor a mayor (en el ejemplo el de Ana), para comprobar las inversiones simplemente hay que pasarle un array con el ranking del otro usuario (en el ejemplo de Juan), y se le indica la posición inicio y fin del array a analizar.

```
public static int inversions (int [] array, int start, int end){

……................................

}
```

```
// mezcla y ordena los elementos de las dos mitades y devuelve el número de inversiones entre ellas
// inicio1, inicio2: primera posición de la primera y segunda mitad respectivamente
// fin1 y fin2: última posición de la primera y segunda mitad respectivamente.

private static int mergeCount(int[] aux, int inicio1, int fin1, int inicio2, int fin2) {
    int i = inicio1;
    int j = inicio2;
    int cont=0;
    int pos = inicio2-inicio1;  //número máximo de inversiones
        ArrayList<Integer> temp = new ArrayList<>();
        while (i <= fin1 && j <= fin2) {
            if (aux[i] <= aux[j]) {
                temp.add(aux[i++]);
                pos--;
            } else {
                temp.add(aux[j++]);
                cont += pos;
            }
        }
        while (i <= fin1) {
            temp.add(aux[i++]);
        }
        while (j <= fin2) {
            temp.add(aux[j++]);
        }
        for (i = inicio1; i <= fin2; i++) {
            aux[i] = temp.remove(0);
        }
        return cont;
    }
```